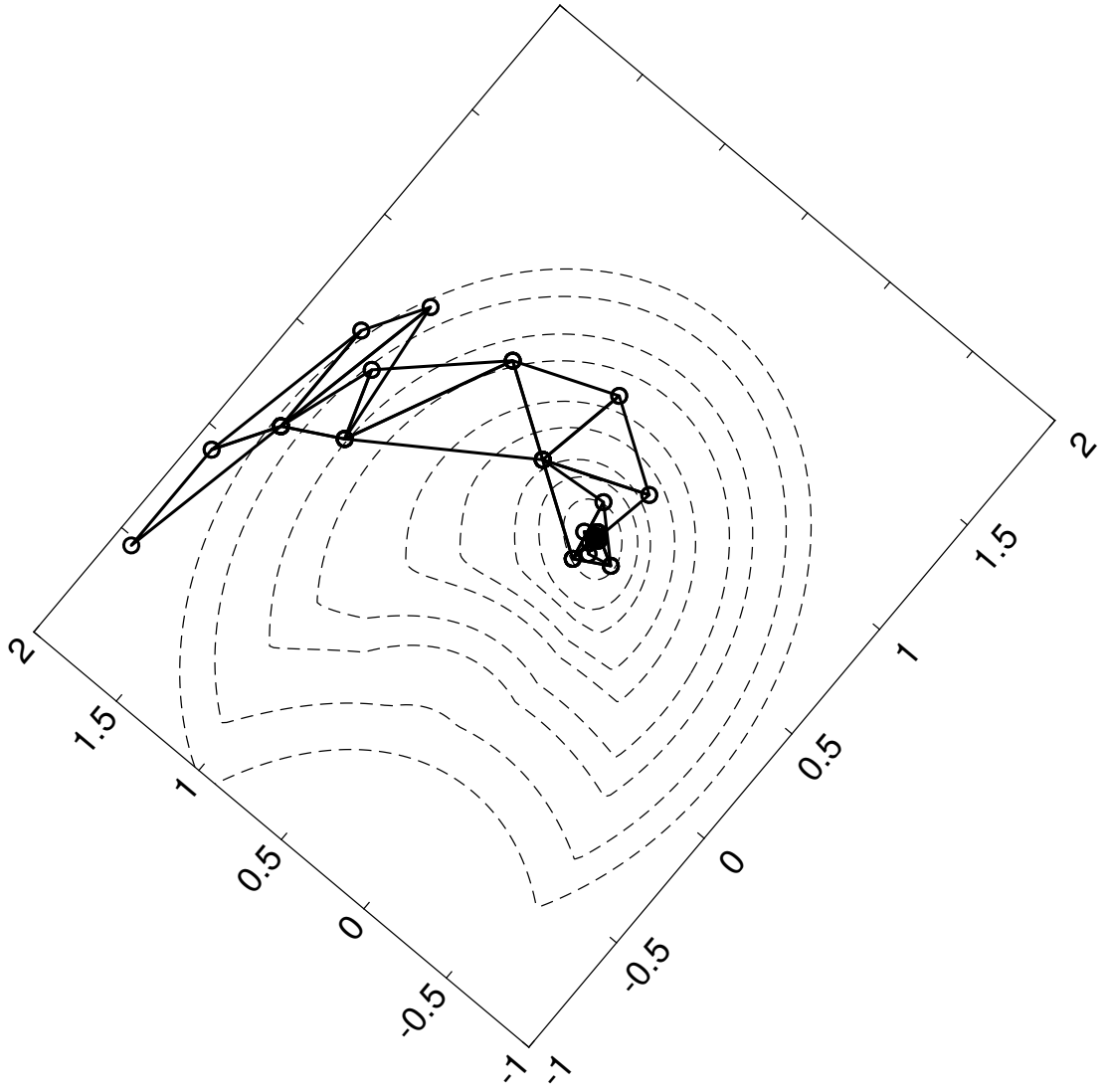
Grzegorz Sieklucki

Optymalizacja nap?du pojazdu na przyk?adzie MINI Cooper



26 października 2022

Spis treści

Rozdział 1. Wprowadzenie 5

Rozdział 2. Optymalizacja bez warunków ograniczających 7

2.1. Wprowadzenie 7
2.2. Optymalizacja w kierunku ($x \in \mathbb{R}$) 7
2.2.1. Ćwiczenia z metod 1. wymiarowych 7
2.3. Wielowymiarowe metody obliczeniowe optymalizacji 8
2.3.1. Metoda Simplex 8
2.3.2. Ćwiczenia z metody Simplex 10
2.3.3. Ćwiczenie z metod randomizowanych 11

Rozdział 3. Zastosowania metod obliczeniowych optymalizacji 13

3.1. Projektowanie napędu pojazdu elektrycznego EV 13
3.1.1. Model ruchu pojazdu elektrycznego 13
3.1.2. Modelowanie ruchu pojazdu 15
3.1.3. Poprawa osiągów pojazdu Tesla model S 19
3.1.4. Projektowanie napędu dla pojazdu Mini Cooper SE 22
3.1.5. Ćwiczenie 30
3.1.6. Projekt EV 31

Bibliografia 33

Dodatek – optymalizacja osiągów i zasięgu EV - pojemność akumulatorów 33

Rozdział 1

Wprowadzenie

Ćwiczenia zostały przygotowane na podstawie artykułu (Sieklucki 2021), chociaż prawda jest inna, to najpierw były ćwiczenia, a później artykuł.

W ćwiczeniu wykorzystywana jest metoda optymalizacji Simplex, w celu określenia charakterystyki mechanicznej silnika orza przełożenia przekładni, aby pojazd elektryczny posiadał zadane osiągi. Jako przykład wykorzystano dane dla pojazdu MINI Cooper SE.

Rozdział 2

Optymalizacja bez warunków ograniczających

2.1. Wprowadzenie

Definicję **optymalizacji** można określić na kilka sposobów:

1. Optymalizacja - działanie mające na celu zwiększenie efektywności.
2. Optymalizacja to wyznaczenie najlepszego rozwiązania i najczęściej jest to minimum funkcji (funkcjonału).
3. Optymalnie znaczy najlepiej.
4. Decyzja optymalna, to taka od której odstępstwo oznacza gorzej.

Optymalizację dzieli się na dwie grupy:

1. Optymalizacja statyczna - poszukiwanie minimalnej wartości funkcji i odpowiadającej jej wartości argumentu x^* w zbiorze liczb rzeczywistych ($x \in \mathbb{R}^n$, gdzie n jest wymiarem przestrzeni optymalizacji).
2. optymalizacja dynamiczna - poszukiwanie najlepszej funkcji w zbiorze funkcji dopuszczalnych.

2.2. Optymalizacja w kierunku ($x \in \mathbb{R}$)

W poszukiwaniu minimum w kierunku (który wyznaczy inna metoda wielowymiarowa, np. najszybszego spadku) można wykorzystywać różne metody jednowymiarowe, tj. bisekcji, złotego podziału, Fibonacciego, Newtona, aproksymacji kwadratowej. W ćwiczeniu skupiono się na analizie pracy trzech pierwszych metod.

2.2.1. Ćwiczenia z metod 1. wymiarowych

W opracowaniu dokonano modyfikacji plików optymalizacji jednowymiarowej ze strony mathworks.com (zakładka File Exchange), których twórcą jest: *Indian Institute of Technology Madras*. Pliki te to procedury optymalizacji, które zostały uzupełnione i rozszerzone w celach edukacyjnych. Wywoływanie różnych metod optymalizacji jednowymiarowej w zakresie (a,b) dla różnych funkcji y (*funkcja.m*):

```
clear
a=0;b=6; %Zakres poszukiwania minimum lokalnego
y=@(x) -0.0041*x.^7+0.0535*x.^6-0.19*x.^5-0.06*x.^4+1.04*x.^3-0.25*x.^2-1.6*x+1
%y=@(x) (x-2).^2
%y=@(x) 0.65-0.75./(1+x.^2)-0.65*x.*atan(1./x); a=0,b=6;
clf
[x1,xu,iter]=GoldenSectionMethod1(a,b,6,y)
%[x1,xu,iter]=FibonacciMethod1(a,b,6,y)
%[xmin,fmin]=Bisection1(a,b,6,y)
```

W katalogu *R1*, oprócz *funkcja.m*, znajdują się również metody optymalizacji, które są wybierane poprzez usunięcie znaku '%'. Należy dla każdej funkcji y:

1. Zaobserwować jak przebiega proces zawężania zakresu, w którym występuje minimum, dla poszczególnych funkcji y w skrypcie.

2. Wyniki należy przeanalizować i zwiększyć ilość kroków z 6. do 50., jakie uzyskane zostaną wartości optymalne?
3. Dla pierwszej funkcji zwiększyć zakres poszukiwania minimum na (-2,6), czy zostanie wyznaczone minimum globalne jakąkolwiek z metod?

2.3. Wielowymiarowe metody obliczeniowe optymalizacji

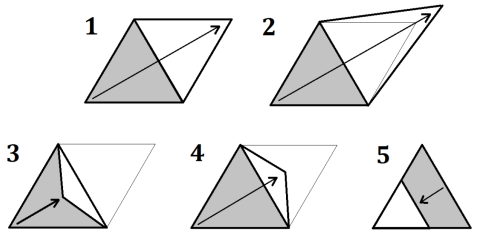
Poszukiwanie minimum funkcjonału można przeprowadzić stosując różne metody, które charakteryzują się różną zbieżnością oraz niezbędnymi obliczeniami, np. gradientu funkcjonału. Do najpopularniejszych metod zalicza się:
— Metodę Gaussa-Seidela,
— Metodę najszybszego spadku.
— Metodę Rosenbrocka.
— Metodę Nedlera-Meada (Simplex).
— Metodę Fletchera-Reevsa (gradientu sprzężonego).
— Metodę Powella.
— Metody randomizowane, które wykorzystują liczby pseudolosowe.
W niniejszym rozdziale największy nacisk zostanie położony na metodę Simplex.

2.3.1. Metoda Simplex

Metoda Neldera i Meada lub Simplex zwana również algorytmem ameby jest metodą numeryczną optymalizacji statycznej bez ograniczeń. W metodzie nie wykorzystuje się gradientu funkcji celu, co czyni ją łatwą w zastosowaniu do problemów technicznych, gdzie występują problemy z wyznaczeniem gradientu.

Jako *simplex* rozumie się figurę wypukłą w przestrzeni n wymiarowej, która posiada $n+1$ wierzchołków, Oznacza to, że dla $n = 1$ jest to odcinek, dla $n = 2$ trójkąt, a dla $n = 3$ czworościan. Sama metoda wymaga zdefiniowania początkowego simplexu (na podstawie punktu startowego x_0), a następnie wykonywane są na nim operacje odbicia, rozszerzenia, zawężenia, skrócenia w celu wyznaczenia najmniejszej wartości funkcji celu $f(x)$.

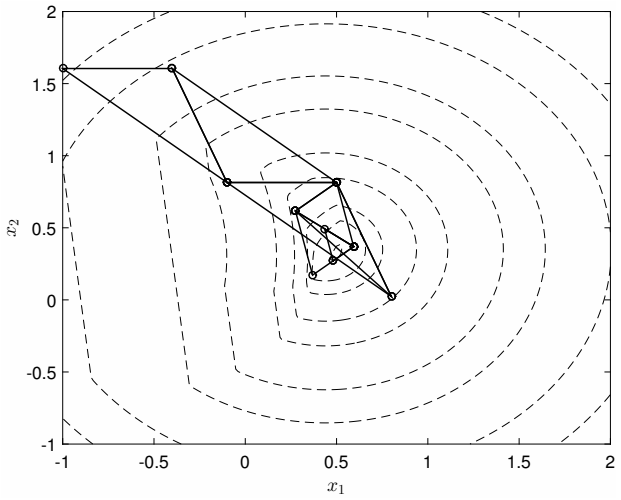
Na rysunku 2.1 przedstawione zostały operacje na simpleksie, które powinny prowadzić do określenia minimum lokalnego funkcji celu.



Rysunek 2.1. Operacje na simpleksie: 1. odbicie, 2. rozszerzenie, 3. kontrakcja do wewnątrz, 4. kontrakcja na zewnątrz, 5. skrócenie

Dla kolejnych operacji przyjmuje się współczynniki: $1, 2, \frac{1}{2}, \frac{1}{2}$. Wszystkie operacje dotyczą wierzchołka dla którego wartość funkcji celu jest największa.

Na rysunku 2.2 przedstawiono proces optymalizacji, który zawiera następujące operacje na simpleksie: inicjalizacja (określenie początkowego simpleksu), odbicie, odbicie, kontrakcja do wewnątrz, kontrakcja do wewnątrz, odbicie i skrócenie.



Rysunek 2.2. Przykładowy proces optymalizacji

W środowisku MATLABmetoda Simplex jest nazwana *fminsearch*, gdzie wyznaczone zostaje minimum funkcji $f(x)$. Metoda ta jest częścią biblioteki *Optimization Toolbox*.

Składnia funkcji zawiera funkcję celu `fun`, warunek (punkt początkowy) `x0` oraz opcje optymalizacji `options` (omówione poniżej):

```
x = fminsearch(fun,x0)
x = fminsearch(fun,x0,options)
[x,fval]  = fminsearch(fun,x0,options)
[x,fval,exitflag] = fminsearch(fun,x0,options)
[x,fval,exitflag,output]  = fminsearch(fun,x0,options)
```

gdzie wartościami zwracanymi przez `fminsearch` są:

— wektor `x` dla minimum lokalnego funkcji `fun`,

— `fval` wartość funkcji celu `fun` w punkcie `x`,

— `exitflag` jest identyfikatorem przyczyny zakończenia obliczeń:

1 – funkcja zbieżna do rozwiązania `x`,

0 – liczba iteracji przekroczyła `options.MaxIter` lub liczba wywołań funkcji przekroczyła `options.MaxFunEvals` (więcej informacji w opisie opcji metody).

-1 – algorytm zakończony przez funkcję wyjścia i przyczyny sa opisane w zmiennej `output`,

— `output` zawiera informacje dotyczące optymalizacji:

`iterations` jest liczbą iteracji,

`funcCount` jest liczbą wywołań funkcji,

`algorithm` w tym przypadku jest: *Nelder-Mead simplex direct search*,

`message` wiadomość wyjściowa.

Opcje dla danego zadania optymalizacji umieszczone są w strukturze `options`, którą najłatwiej jest wypełniać z wykorzystaniem polecenia `optimset`. Dla metody `fminsearch` dostępne są następujące ustawienia:

`Display` wyświetlanie wyników:

`'off'` brak,

'iter' co iterację,
'notify' tylko gdy metoda nie jest zbieżna,
'final' tylko ostatni wynik.

FunValCheck 'on' wyświetla błąd, gdy wartość funkcji jest zespolona, nieskończona (Inf) lub nieokreslona (NaN), domyślnym ustawieniem jest 'off',
MaxFunEvals maksymalna liczba wywołań funkcji celu, domyślnie 200*numberOfVariables,
MaxIter domyslnie 200*numberOfVariables,
OutputFcn zdefiniowana przez użytkownika funkcja wywoływana po każdej iteracji, domyslnie brak,
PlotFcns wyświetlanie graficzne różnych miar postępu algorytmu,
TolFun warunek stopu na podstawie błędu bezwzględnego poprawy rozwiązania, domyślnie 10^{-4},
TolX warunek stopu na podstawie błędu bezwzględnego poprawy rozwiązania, domyślnie 10^{-4}.

UWAGA: w przeciwieństwie do pozostałych metod optymalizacji w metodzie fminsearch warunkiem stopu jest jednoczesne spełnienie TolFun i TolX.

2.3.2. Ćwiczenia z metody Simplex

Funkcją celu, minimum której jest poszukiwane, jest „bananowa dolina Rosenbrocka":

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \tag{2.1}$$

Aby numerycznie wyznaczyć minimum tej funkcji (metodą Simplex) należy wpisać do okan komend Matlaba:

```
fun = @(x)(100*(x(2) - x(1)^2)^2 + (1 - x(1))^2);
options = optimset('OutputFcn',@bananaout,'Display','off');
x0 = [-1.9,2];
[x,fval,eflag,output] = fminsearch(fun,x0,options);
title 'Rosenbrock solution via fminsearch'
Fcount = output.funcCount;
disp(['Number of function evaluations for fminsearch was ',num2str(Fcount)])
disp(['Number of solver iterations for fminsearch was ',num2str(output.iterations)])
```

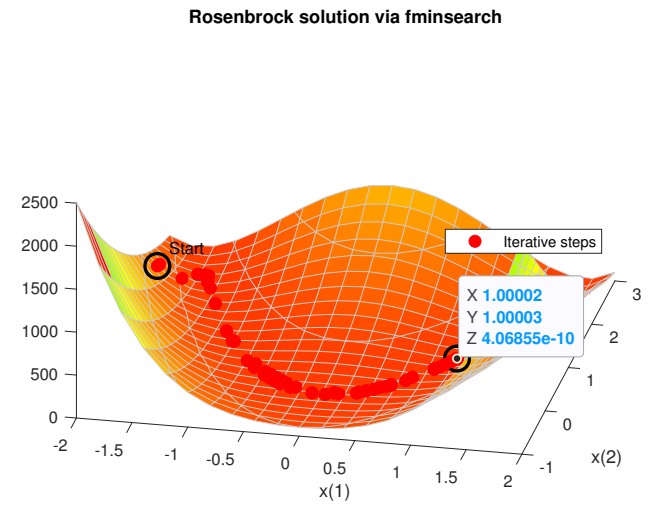
Wówczas uzyskuje się wynik jak na na rysunku 2.3.

Dalsza część ćwiczeń dotyczy demonstracji obliczeń w metodzie Simplex i wykorzystano tu programy na podstawie materiałów Henri P. Gavin -*Example of Running Constrained Miminization Codes*. Duke University Department of Civil and Environmental Engineering 2012. Wszystkie potrzebne pliki znajdują się w katalogu *R2* i należy postępować zgodnie z następującymi punktami:

1. Należy kilkakrotnie uruchomić skrypt *optim_ex.m*, gdzie odbywa się optymalizacja metodą simplex – można wybrać różne punkty startowe w tym pliku (linie 4-7), pierwszy warunek początkowy *p_init* jest dobierany losowo. Należy zwrócić uwagę jak punkt startowy wpływa na rozwiązanie przy ograniczonej liczbie kroków. Minimalizowaną funkcją jest (2.1).
2. W pliku *optim_ex.m* należy zmienić *optim_example_analysis* na *optim_example_analysis1*, w tym przypadku autor zastosował metodę simplex do minimalizacji funkcji

$$f(x) = (x_1 - c_1)^2 + (x_2 - c_2)^2$$

w zbiorze dopuszczalnym (z ograniczeniami nierównościowymi). Wykres poziomicowy przedstawia zmodyfikowaną funkcję celu (**??**) i kolejne kroki optymalizacji. Szczegóły znajdują się w pliku *optim_example_analysis1.m*.

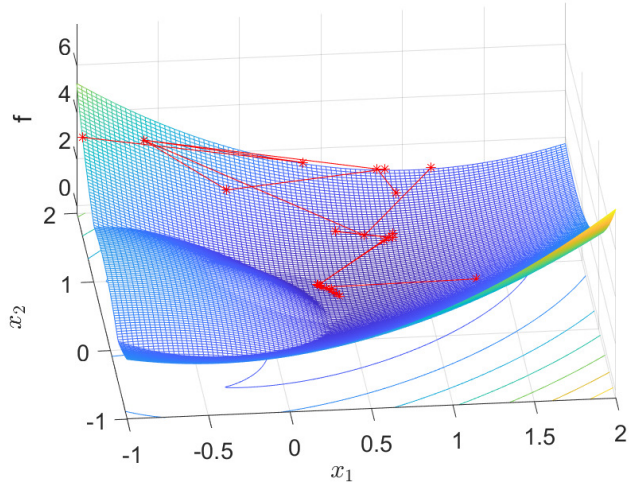


Rysunek 2.3. Przykładowy proces optymalizacji dla (2.1)

2.3.3. Ćwiczenie z metod randomizowanych

W tej części również będą wykorzystywane powyższe pliki, ale w pliku *optim_ex.m* należy zablokować linię 18 i odblokować 17. Następnie można uruchomić obliczenia. W metodzie tej, w punkcie obliczeniowym, kierunek obliczeń i krok są wybierane wielokrotnie w sposób losowy i dalej wybierany jest najlepszy wynik.

Należy uruchomić kilkakrotnie *optim_ex.m*, aby zaobserwować wpływ punktu startowego na uzyskany wynik, przykład przedstawiono na rysunku 2.4.



Rysunek 2.4. Przykładowy proces optymalizacji dla metody randomizowanej

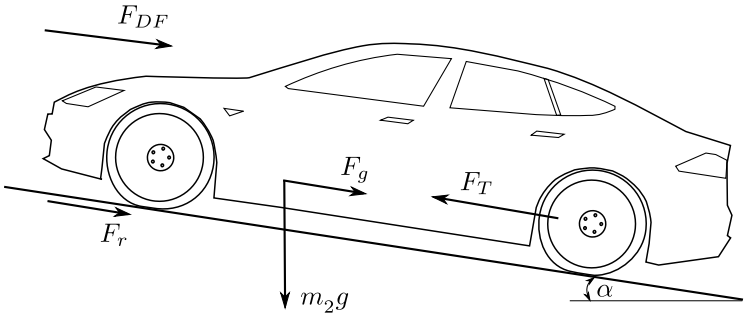
Rozdział 3

Zastosowania metod obliczeniowych optymalizacji

3.1. Projektowanie napędu pojazdu elektrycznego EV

3.1.1. Model ruchu pojazdu elektrycznego

Najważniejsze siły (opory ruchu i siła trakcyjna – silnika głównego) przeniesione na koła pojazdu zostały zaprezentowane na rysunku 3.1 (Gillespie 1992, Sieklucki 2018).



Rysunek 3.1. Siły działające na pojazd

Siła trakcyjna – przeniesienie napędu (ΔM - moment niezbędny do wprawienia w ruch przekładni, d_w - średnica koła, i - przełożenie przekładni mechanicznej):

$$F_T = (M_e - \Delta M)\frac{2i}{d_w} \tag{3.1}$$

Całkowite opory tarcia F_r są modelowane w najprostszy sposób, tzn. jest to sumaryczny opór wszystkich kół (Gillespie 1992):

$$F_r = f_r m_2 g, \quad f_r = f_0\left(1 + \frac{v}{161}\right) \tag{3.2}$$

gdzie v jest prędkością pojazdu w km/h, $f_0 = 0,01 \div 0,02$ dla utwardzonej nawierzchni (asfalt). W literaturze można odszukać szereg zależności określających opory tarcia opona-podłoże i można je wykorzystać do przybliżonego określenia tej siły.

Opór aerodynamiczny (siła oporu powietrza):

$$F_{DF} = \frac{1}{2}\varrho\, C_d A (v - v_w)^2 \tag{3.3}$$

gdzie gęstość powietrza na poziomie morza w temperaturze 15^oC wynosi $\rho = 1.225$kg/m^3, natomiast v_w jest prędkością wiatru.

W modelu matematycznym ruchu pojazdu powinno być również uwzględnione nachylenie terenu, czyli siła związana z przyciąganiem ziemskim – rozkład sił na poszczególne osie pojazdu został pominięty, wówczas tę siłę określa się jako:

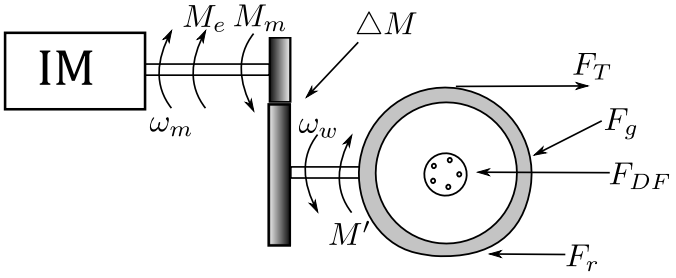
$$F_g = m_2 g \sin(\alpha) \tag{3.4}$$

Zastosowanie 2. zasady dynamiki Newtona prowadzi do wyrażenia:

$$m \underbrace{\frac{dv(t)}{dt}}_{a(t)} = F_T(t) - \underbrace{(F_{DF}(t) + F_r(t) + F_g(t))}_{F'(t)} \tag{3.5}$$

Z uwagi na fakt, że siła oporu powietrza (3.3) jest wyrażeniem nieliniowym, więc równanie różniczkowe zwyczajne (3.5) jest również równaniem nieliniowym. Z tego powodu najlepszym sposobem określenia osiągów danego pojazdu jest zastosowanie metod numerycznych w rozwiązaniu równania (3.5).

Siły działające na koła pojazdu i momenty przeniesione na wał silnika zostały przedstawione na rysunku 3.2. Przekształcenie wypadkowej siły obciążającej F' na moment obciążenia silnika M_m



Rysunek 3.2. Uproszczone przeniesienie napędu z silnika na koła pojazdu

przeprowadza się z wykorzystaniem przełożenia przekładni:

$$M_m = \Delta M + \frac{M'}{i}, \quad M' = \frac{F' d_w}{2} \tag{3.6}$$

i zastosowanie 2. zasady dynami Newtona dla układu wirującego prowadzi do:

$$J \frac{d\omega_m(t)}{dt} = M_e(t) - M_m(t) \tag{3.7}$$

gdzie J jest całkowitym momentem bezwładności przeniesionym na stronę silnika. Moment ten wyznacza się na podstawie porównania energii kinetycznej dla ruchu prostoliniowego i obrotowego:

$$\frac{m_x v^2}{2} = \frac{J_x \omega_m^2}{2}$$

Również na podstawie powyższej zależności można wyznaczyć ekwiwalent masy (masę równoważną), czyli przeniesie dowolnego momentu bezwładności J_x silnika na karoserię pojazdu.

W pojeździe można określić 3 magazyny energii: wirnik silnika (moment bezwładności J_1) i masa pojazdu wraz z kierowcą (m_2), wówczas odpowiednie ekwiwalenty masy i momentu bezwładności wynoszą:

$$m_1 = J_1 \frac{4i^2}{d_w^2}, \qquad J_2 = m_2 \frac{d_w^2}{4i^2} \tag{3.8}$$

wartości m_1, J_1 są znacznie mniejsze od m_2, J_2.

Kolejne magazyny energii to pozostałe elementy obrotowe: koła (felgi i opony), tarcze hamulcowe, wały, itp. Biorąc pod uwagę każdą obracającą się masę m_i o promieniu r_i, można określić jej moment bezwładności lub określić masę równoważną (dla ruchu poziomego) tej samej energii kinetycznej.

$$J_i = m_i r_i \tag{3.9}$$

J_i to każdy element obrotowy w przeniesieniu napędu. Stąd z zasady zachowania energii wynika:

$$\frac{m_{3i} v^2}{2} = J_i \frac{\omega^2}{2} \tag{3.10}$$

gdzie m_{3i} jest ekwiwalentem masy obliczonej przeniesionym na nadwoziu pojazdu. Jeżeli v jest prędkością samochodu, a promień koła wynosi $r_w = d_w/2$, to równanie (3.10) prowadzi:

$$\frac{m_{3i}\omega^2 r_w^2}{2} = \frac{m_i r_i^2 \omega^2}{2} \quad \Longrightarrow m_{3i} = m_i \frac{r_i^2}{r_w^2} \tag{3.11}$$

gdzie r_i jest promieniem wirującego elementu.

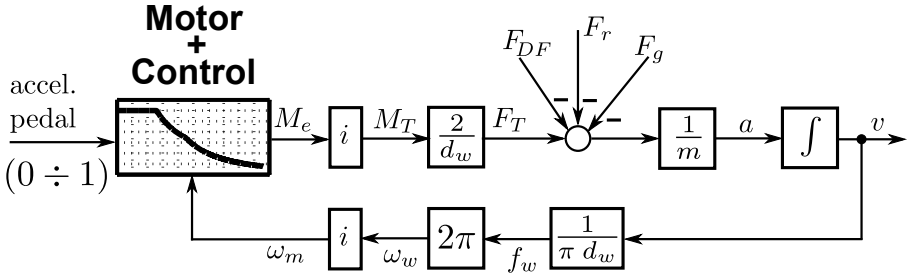
Na przykład cztery koła, każde o wadze 20 kg, należy dodać do masy pojazdu jako 80 kg, natomiast cztery tarcze hamulcowe, każda o wadze 3 kg i średnicy o połowę mniejszej od średnicy koła, należy ekwiwalent zmniejszyć czterokrotnie, tj. dodać 3 kg do nadwozia pojazdu.

W związku z tym w modelu matematycznym można pominąć elementy obrotowe o małej średnicy i małej masie. Podsumowując, całkowity moment bezwładności zredukowany do silnika i całkowita masa zredukowana do nadwozia pojazdu są następujące:

$$J = J_1 + J_2 + \frac{\sum_i J_i}{i^2}, \qquad m = m_1 + m_2 + \sum_i m_{3i} \tag{3.12}$$

Równania (3.5) i (3.7) opisują ten sam elektromechaniczny system, ale w różnych układach odniesienia.

Powyższe równania można przedstawić w postaci schematu blokowego przetwarzania sygnałów – rysunek 3.3. Schemat ten można przekształcić wykorzystując zależność (3.6) i wówczas siły oporu są przeniesione na stronę silnika, co ułatwia proces projektowania napędu pojazdu (rys. 3.4)

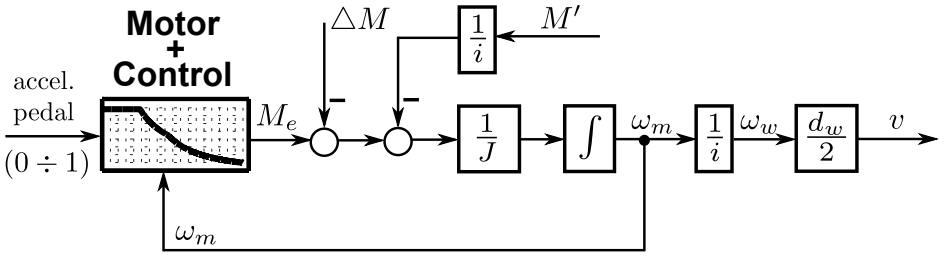


Rysunek 3.3. Uproszczone sterowanie napędem pojazdu elektrycznego

Blok **Motor+Control** symbolizuje charakterystykę mechaniczną napędu.

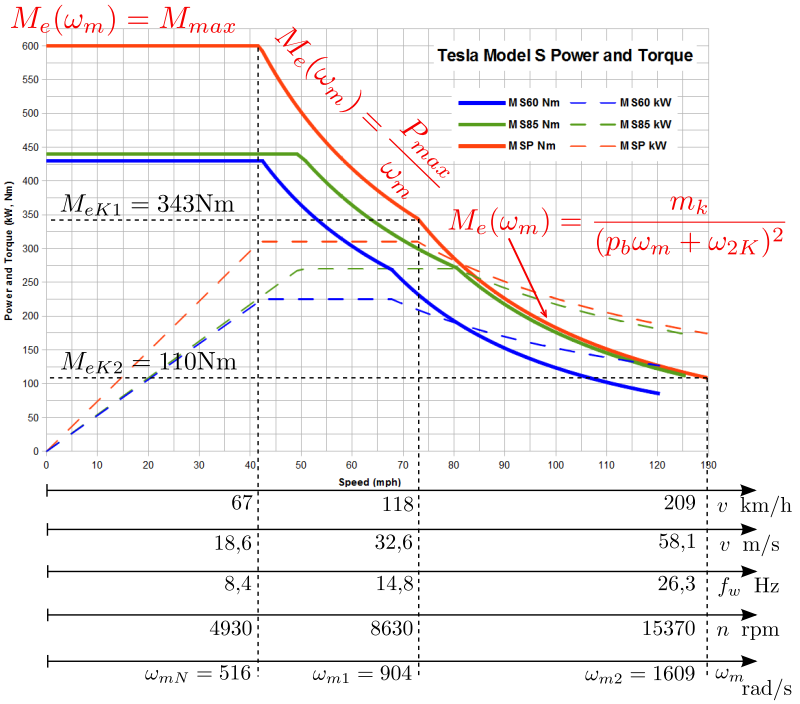
3.1.2. Modelowanie ruchu pojazdu

W bloku **Motor+Control** zapisana jest charakterystyka mechaniczna silnika (rys. 3.3 i 3.4), czyli zależność momentu obrotowego (elektromagnetycznego) od prędkości kątowej. Realizacja tej charakterystyki odbywa się przez odpowiednie sterowanie (regulację momentu) silnikiem elektrycznym



Rysunek 3.4. Uproszczone sterowanie napędem pojazdu elektrycznego – model przekształcony

(Sieklucki 2018). Przykładowe charakterystyki dla Tesla model S przedstawiono na rysunku 3.5. Analizie poddano tu, charakterystykę dla wersji 85+ (kolor czerwony), wówczas samochód charak-



Rysunek 3.5. Charakterystyki mechaniczne silników z pojazdu Tesla model S 2012r.

teryzuje się następującymi osiągami:
— 0-60mph (96,6km/h): 4,0s,
— 1/4-mili (402,5m): 12,7s i prędkość: 107.8 mph (173,6 km/h).
Pod charakterystyką zamieszczono wartości charakterystyczne (pojazdu i silnika) dla odpowiednich zakresów charakterystyki, gdzie f_w jest częstotliwością obracających się kół, a n i ω_m są prędkością silnika elektrycznego wyrażoną w różnych jednostkach (rpm to obroty na minutę).

Wynika stąd, że charakterystyka pracy silnika jest nieliniowa, więc schematy z rysunków 3.3 i 3.4 przedstawiają w postaci graficznej nieliniowe równania różniczkowe. Z tego powodu najlepszą metodą sprawdzenia osiągów pojazdu jest modelowania komputerowe. Do tego celu konieczna jest znajomość podstawowych parametrów pojazdu:
— masa pojazdu z kierowcą $m \approx 2200$kg,

— współczynnik oporu powietrza $C_d = 0.24$, powierzchnia czołowa pojazdu $A = 2,4\text{m}^2$,
— przełożenie przekładni mechanicznej $i = 9,73$.
— opony: P245/45R19, czyli średnica kół wynosi $d_w = 0.703\text{m}$.

Z uwagi na fakt, że badając rozpędzanie się pojazdu, nie spodziewamy się żadnych oscylacji metoda Eulera (najprostsza) do całkowania równań różniczkowych będzie tu wystarczająca. Rozważając problem ruchu dla naturalnego układu odniesienia, tzn. Ziemi (eq. (3.5)), przyspieszenie i prędkość pojazdu określa się jako:

$$a(t) = \frac{F_T(t) - F'(t)}{m}, \quad \frac{dv(t)}{dt} = a(t) \tag{3.13}$$

Zastosowanie metody Eulera (gdzie T_s jest nazywane krokiem całkowania)

$$\frac{dv(t)}{dt} \approx \frac{v(t+T_s) - v(t)}{T_s}$$

prowadzi do następującego równania różnicowego

$$v(t+Ts) = v(t) + T_s a(t)$$

które można zapisać w postaci algorytmicznej

$$v(k+1) = v(k) + T_s a(k) \tag{3.14}$$

gdzie k jest krokiem obliczeniowym.

Wyznaczenie położenia pojazdu $s_{pos}(t) = \int_0^t v(\tau)d\tau$ także wymaga całkowania numerycznego. Wówczas na podstawie metody Eulera uzyskuje się

$$s_{pos}(k) = T_s v(k) + s_{pos}(k-1) \tag{3.15}$$

Autorzy zdają sobie sprawę z faktu, że powyższe rozdzielenie równań jest nieprawidłowe pod kątem metod numerycznych, ale ilustruje ono algorytm numerycznego całkowania zadanego przebiegu. Prawidłowo problem rozwiązania równań ruchu powinien być zastosowany do:

$$\begin{cases} \dfrac{dv(t)}{dt} = a(t) \\ \dfrac{ds_{pos}(t)}{dt} = v(t) \end{cases}$$

Program realizujący modelowanie pojazdu z uwzględnieniem charakterystyki silnika został zamieszczony poniżej:

```
01: clear
02: g=9.81; %gravitational acceleration
03: g_i=9.73;%gear ratio
04: mass=2245; % mass=car+driver+m_1
05: d_w=(19*25.4+2*245*0.45)/1000;  %Diameter of wheels P85
06: %d_w=0; %Diameter of wheels P85+
07: Pn=310000; Mmax=600;pb=2;
08: wI=516; %end of mode I
09: wII=904; % end of mode II
10: wIII=1920; %max. of IM
11: w_2k=33.1; m_k=1.1627e+09;
12: %-----drag force
13: rho=1.225; % Density of air kg/m3
14: Cd=0.24; % drag coefficient
15: A=2.4; % the reference area.
16: %%---- IM curve
```

```
17: wm_vect=0:0.5:1920;
18: for i=1:length(wm_vect)
19:   Me(i)=(wm_vect(i)<=wI)*Mmax...
+(wm_vect(i)>wI&&wm_vect(i)<=wII)*Pn/wm_vect(i)...
+(wm_vect(i)>wII)*m_k/(pb*wm_vect(i)+w_2k)^2;
20: end
21: P=Me.*wm_vect;
22: fig1=figure(1); hold off
23: plot(wm_vect,Me,'k','LineWidth',1.5); hold on
24: plot(wm_vect,P/1000,'k--','LineWidth',1.5);
25: xlabel('$\omega_m$','FontSize',15,'Interpreter','latex')
26: ylabel('$M_e, P$','FontSize',15,'Interpreter','latex')
27: leg1=legend('$M_e$   Nm','$P$   kW');
28: set(leg1,'FontSize',15,'Interpreter','latex');
29: set(gca,'FontSize',12);
30: grid on
31: %%------ Numerical methods
32: t_step=0.05; t_end=33; %interval and simulation time
33: t=0:t_step:t_end;
34: wm=zeros(1,length(t))+0.01;  %initial value of wm
35: v_ms=zeros(1,length(t))+0.0001;   %initial value of v
36: F_g=0*wm;   % grade resistance
37: clear Me;
38: dM=18; %load torque in gear
39: for k=1:length(t)-1
40:   Me(k)=(wm(k)<=wI)*Mmax...
+((wm(k)>wI)&&(wm(k)<=wII))*Pn/wm(k)...
+(wm(k)>wII)*m_k/(pb*wm(k)+w_2k)^2;
41:   F_T(k)=2/d_w*g_i*(Me(k)-dM);
42:   F_r(k)=mass*g*0.01*(1+v_ms(k)*3.6/161);
43:   F_DF(k)=1/2*rho*Cd*A*v_ms(k)^2;
44:   a(k)=1/mass*(F_T(k)-F_r(k)-F_DF(k)-F_g(k));
45:   if (wm(k)>wIII) a(k)=0; end
46:   v_ms(k+1)=v_ms(k)+t_step*a(k); % speed m/s
47:   wm(k+1)=2*pi*g_i/(pi*d_w)*v_ms(k);
48: end
50: Me(k+1)=Me(k); a(k+1)=a(k); %complement the vectors
51: F_T(k+1)=F_T(k);F_DF(k+1)=F_DF(k);F_r(k+1)=F_r(k);
52: s_pos=zeros(1,length(t));
53: for k=2:length(t) %replace: s_pos=t_step*cumsum(v_ms);
54:   s_pos(k)=t_step*v_ms(k)+s_pos(k-1);
55: end;
56: %%---- VISUALIZATION------
57: figure(2)
58: v_kmh=v_ms*3.6; % convert m/s to kmh
59: %p1=plot(t,F_T,t,F_DF,t,F_r,t,F_DF+F_r);
60  % yl=ylabel('Forces');
61: %leg1=legend('F_T','F_{DF}','F_r','F_{DF}+F_r');
62:   %set(leg1,'FontSize',15)
63: %p1=plot(t,a);
64: %p1=plot(t,wm);
65: p1=plot(t,v_kmh,'k',[t(1) max(t)],[96.6 96.6],'r');
66:    yl=ylabel('v/kmh');
67: %p1=plot(t,s_pos,'k',[t(1) max(t)],[402.5 402.5],'r');
68:   %yl=ylabel('s_{pos}/m');
69: grid; axis tight
70: set(p1,'LineWidth',1.5)
71: xl=xlabel('t');
72: set(xl,'FontSize',15);set(yl,'FontSize',15);
73: set(gca,'FontSize',12);
```

W kolejnych liniach powyższego skryptu znajduje się:

1÷ 15 parametry pojazdu i strefy charakterystyki sterowania silnikiem indukcyjnym,

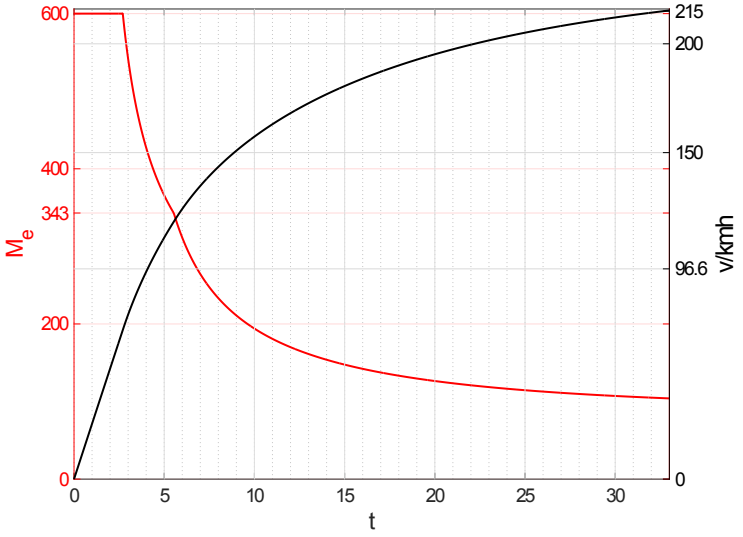
17÷ 30 wyświetlenie charakterystyki moment i moc silnika w funkcji prędkości kątowej silnika,

40 zapisana jest charakterystyka silnika elektrycznego, w tym przypadku jest to silnik indukcyjny.

32÷ 55 symulacja ruchu (metoda Eulera), gdzie `t_step`= T_s, a `t_end` jest czasem symulacji,

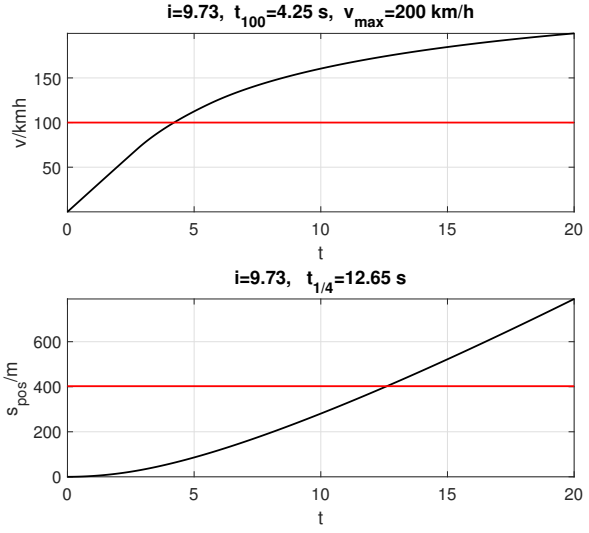
57÷ 73 wizualizacja wyników: należy odznaczać odpowiednią sekcję, aby uzyskać określony przebieg. Dla prędkości i położenia zaznaczono wartości osiągów pojazdu (0-60mph i 1/4-mili).

Na rysunku 3.6 przedstawiono wyniki modelowania: przebieg momentu i prędkości pojazdu.



Rysunek 3.6. Symulacja silnika Tesli model S P85+

Na rysunku 3.7 przedstawiono wyniki symulacji, skąd można określić osiągi pojazdu.



Rysunek 3.7. Osiągi Tesli model S P85+

3.1.3. Poprawa osiągów pojazdu Tesla model S

Zadanie optymalizacji polega na wyznaczeniu przełożenia przekładni mechanicznej i, tak aby uzyskać lepsze przyspieszenie do 100km/h lub lepszy czas na 1/4 mili lub największą możliwą prędkość maksymalną. Badania zostaną przeprowadzone z wykorzystaniem metody Simplex, której wywołanie znajduje się w skrypcie `optymalizacja_samochodu.m`:

```
%optymalizacja samochodu
przekladnia0=6;
[i] = fminsearch(@tesla,przekladnia0)%optymalizacja przekładni mech.
```

Natomiast funkcja celu znajduje się w pliku `tesla.m`:

```
function [J2]=tesla(przekladnia_i)
%Optymalizacja przekładni mechanicznej
% J - to wskaźnik dla przyspieszenia do 100kmh
%J1 - to wskaźnik dla 1/4 mili
%J2 - max. prędkosc

g=9.81;
mass=2200;% masa pojazdu z kierowca
%d_k=(19*25.4+2*245*0.45)/1000; %srednica kola dla P85
d_k=(21*25.4+2*265*0.35)/1000; %srednica kola dla P85+
Pn=310000; Mmax=600;pb=2;
wI=516; %koniec strefy I
wII=904; % koniec strefy II
wIII=1920; %maksymalna predk. silnika
w_2k=33.1;
m_k=1.1627e+09;
%-----opory
rho=1.225; % gestość powietrza kg/m3
Cd=0.24; % wsp. oporu powietrza
A=6.2/((3.281)^2)/Cd; % płaszczyzna natarcia-konwersja Cd*A=6.2ft^2

%% Przyspieszenie
t_step=0.05; t_end=200;

t=0:t_step:t_end;
wm=zeros(1,length(t))+0.01;      %0.01 to wartosc poczatkowa
v_ms=zeros(1,length(t))+0.0001;   %predkosc w m/s
F_g=0*wm;    % nachylenie terenu
clear Me;
dM=12; %load torque in gear
for k=1:length(t)-1
 Me(k)=(wm(k)<=wI)*Mmax+((wm(k)>wI)&&(wm(k)<=wII))*Pn/wm(k)+(wm(k)>wII)*m_k/(pb*wm(k)+w_2k)^2;
 F_T(k)=2/d_k*przekladnia_i*(Me(k)-dM);    % sila trakcyjna
 F_r(k)=mass*g*0.01*(1+v_ms(k)*3.6/161);
 F_DF(k)=1/2*rho*Cd*A*v_ms(k)^2;
 a(k)=1/mass*(F_T(k)-F_r(k)-F_DF(k)-F_g(k));
 if (wm(k)>wIII) a(k)=0; end
 v_ms(k+1)=v_ms(k)+t_step*a(k); % predkosc w m/s
 wm(k+1)=2*pi*przekladnia_i/(pi*d_k)*v_ms(k);
end
Me(k+1)=Me(k); a(k+1)=a(k);%uzupelnienie o brakujacy element
F_T(k+1)=F_T(k);F_DF(k+1)=F_DF(k);F_r(k+1)=F_r(k);
s_pos=zeros(1,length(t));
for k=2:length(t) % w Matlabie: s_pos=t_step*cumsum(v_ms);
    s_pos(k)=t_step*v_ms(k)+s_pos(k-1);
end;

%%----FUNKCJE CELU:
%przyspieszenie do 100km/h
lt=length(t);
t_end=t(lt);
to=t_end;
n=1;                    %indeksowanie
while(v_ms(n)<100/3.6)&&(n<lt)
```

```
55      to=t_end;
56      n=n+1;
57  end
58  to=t(n);
59  J=to;  %=========wsk. jakości -- funkcja celu dla 100kmh
60
61  % wyznaczenie wskaźnika na 1/4mili=402.5m
62  lt=length(t);
63  t_end=t(lt);
64  t14=t_end;
65  n=1;                    %indeksowanie
66  while(s_pos(n)<402.5)&&(n<lt)
67      t14=t_end;
68      n=n+1;
69  end
70  t14=t(n);
71  J1=t14;
72
73  % wyznaczenie wskaźnika dla max. prędkości
74  lt=length(t);
75  v_max=v_ms(lt);
76  J2=-v_max
77
78  %% wizualizacja
79  figure(5);
80  v_kmh=v_ms*3.6; % Multiply by 3.6 to convert m/sec to kph
81  subplot(211)
82  %p1=plot(t,v_ms,'k',[t(1) t(length(t))], [96.6 96.6]/3.6,'r');yl=ylabel('v/ms');
83  p1=plot(t,v_kmh,'k',[t(1) t(length(t))], [100 100],'r');yl=ylabel('v/kmh');
84  grid; axis tight
85  wynik=sprintf('i=%0.2f,  t_{100}=%0.2f s,  v_{max}=%0.0f km/h',przekladnia_i, to,v_max*3.6);
86  title(wynik)
87  set(p1,'LineWidth',1.5)
88  xl=xlabel('t');
89  set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
90  subplot(212)
91  p1=plot(t,s_pos,'k',[t(1) t(length(t))], [402.5 402.5],'r');yl=ylabel('s_{pos}/m');
92  grid; axis tight
93  wynik=sprintf('i=%0.2f,   t_{1/4}=%0.2f s',przekladnia_i, t14);
94  title(wynik)
95  set(p1,'LineWidth',1.5)
96  xl=xlabel('t');
97  set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
```

W kolejnych liniach kodu można wyróżnić:

1 jako wartość funkcji celu można wybrać jeden z trzech zdefiniowanych wskaźników, które obliczane są w liniach 50÷76,

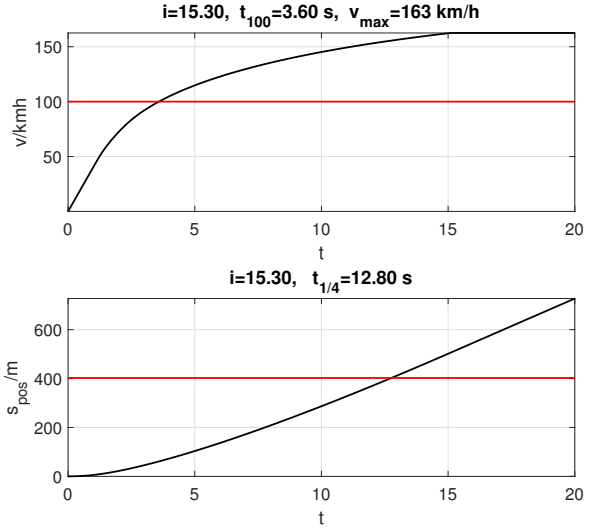
23 ustawienie czasu symulacji,

31÷46 symulacja.

50÷ 76 wyznaczanie funkcji celu zgodnie z opisem w pliku, należy zwrócić uwagę, że wskaźnik J_2 jest przyjęty jako $-v_{max}$, ponieważ funkcja `fminsearch` poszukuje minimum, to zadanie maksymalizacji prędkości zamienia się na minimalizację,

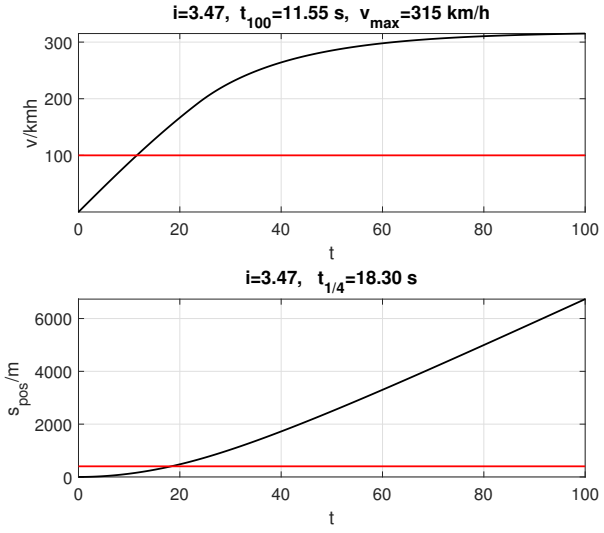
79÷ 97 wizualizacja wyników, gdzie zamiast przyspieszenia 0-60mph przyjęto 0-100km/h.

Przykładowo podano wyniki (rys. 3.8) dla poprawy przyspieszenia od 0 do 100km/h. Widać, że przełożenie $i = 15,3$ (w oryginale jest 9,73) zapewnia czas osiągnięcia 100km/h wynoszący 3,6s, ale prędkość maksymalna zmaleje do 163km/h (10% poprawa). W podobny sposób



Rysunek 3.8. Osiągi Tesli model S P85+ po dokonaniu optymalizacji przekładni dla najlepszego przyspieszenia

można postąpić dla pozostałych dwóch wskaźników i wyniki optymalizacji przekładni dla prędkości maksymalnej zamieszczono na rysunku 3.9. W tym zadaniu konieczne było wydłużenie symulacji do

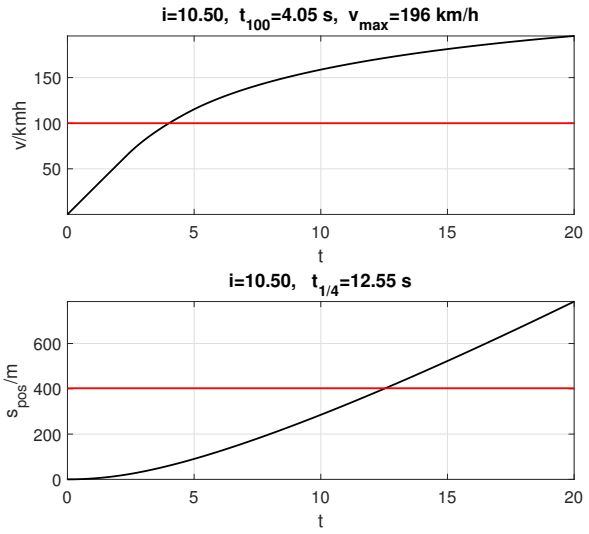


Rysunek 3.9. Osiągi Tesli model S P85+ po dokonaniu optymalizacji przekładni dla prędkości maksymalnej

100s i pojazd uzyskuje prędkość 315km/h dla przełożenia $i = 3,47$. Wynik dla wyścigu na 1/4 mili zamieszczono na rysunku 3.10. W tym przypadku widać nieznaczna poprawę osiągu.

3.1.4. Projektowanie napędu dla pojazdu Mini Cooper SE

W tej części rozważany będzie dobór silnika o dwustrefowej regulacji prędkości oraz przełożenia przekładni mechanicznej. Przedmiotem jest pojazd Mini Cooper SE o następujących parametrach:
— masa pojazdu z kierowcą $m \approx 1365 + 85$kg,



Rysunek 3.10. Osiągi Tesli model S P85+ po dokonaniu optymalizacji przekładni dla wyścigu na 1/4 mili

— współczynnik oporu powietrza $C_d = 0.3$, powierzchnia czołowa pojazdu $A = 2,1\text{m}^2$,
— przełożenie przekładni mechanicznej $i = 8,96$,
— opony: P205/45R17, czyli średnica kół wynosi $d_w = (17 * 25,4 + 2 * 205 * 0,45)/1000 = 0,616$m, czyli obwód koła wynosi 1,935m,
— prędkość maksymalna $v_{max} = 150$km/h, przyspieszenie 0-100km/h wynosi 7,3s.
— ma stronie `https://ev-database.org/car/1409/Mini-Cooper-SE` podano parametry baterii akumulatorów:
  1. pojemność nominalna: 32,6kWh, użyteczna 28,9kWh,
  2. rodzaj akumulatorów: Lithium-ion,
  3. moc ładowania 11kW/AC lub maksymalnie 49kW/DC,
— silnik PMSM 135kW (od 7000 obr/min), moment maksymalny 270Nm (0-1000obr/min). Wiedząc ile wynosi prędkość maksymalna, przekładnia i obwód koła można wyznaczyć maksymalna prędkość silnika elektrycznego:

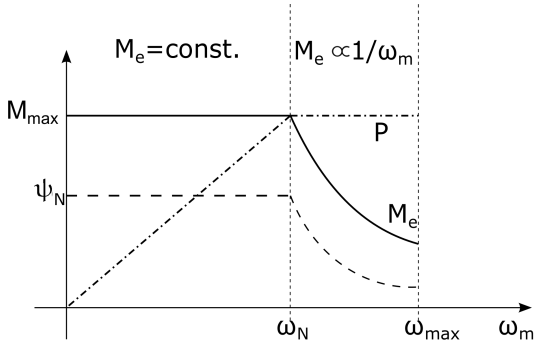
$$\omega_{max} = \frac{v_{max}}{3,6}\frac{1}{\pi d_w}2\pi i = 1212\text{rad/s} \quad \rightarrow n = 11580\text{obr/min}$$

W zadaniu zamiast oryginalnego silnika zastosowano zunifikowaną charakterystykę mechaniczną dla trakcyjnych silników PMSM (Sieklucki, Bisztyga, Zdrojewski, Orzechowski & Sykulski 2014).

W pracy (Plunkett & Plette 1977) omówiono trójstrefową regulację prędkości kątowej silników indukcyjnych, którą następnie uogólniono dla napędów prądu stałego. Opracowanie takie dla pracy przerywanej silników zostało zaprezentowane w pracy (Hindmarsh & Renfrew 1996). Na rysunku 3.11 przedstawiono ogólnie charakterystyki $M = f(\omega)$, $P = f(\omega)$ i $\psi = f(\omega)$ dla pracy ciągłej napędu elektrycznego.

Przedstawione na rysunku 3.11 strefy regulacji oznaczają różne rodzaje pracy silnika elektrycznego.

Strefa 1: oznacza pracę na stały moment $M_e = const$ i wówczas moc czynna przekazywana do maszyny roboczej narasta liniowo wraz ze wzrostem prędkości kątowej ($P_m = M_e\omega_m$). Strefa ta oznacza również pracę z ograniczonym prądem i stałą wartością strumienia w szczelinie powietrznej.



Rysunek 3.11. Teoretyczna trójstrefowa regulacja prędkości kątowej

Strefa 2: oznacza pracę na stałą moc czynną silnika $P_m = P_N = M_e\omega_m$, gdzie moment elektromagnetyczny silnika zmienia się odwrotnie proporcjonalnie do prędkości kątowej silnika ($M_e \propto \frac{1}{\omega_m}$). W strefie tej napięcie zasilania (wirnika dla silnika obcowzbudnego, a stojana dla silników prądu przemiennego) jest równe napięciu znamionowemu U_N i jest to ograniczenie napięciowe. Równocześnie prąd uzwojenia twornika ma wartość znamionową, a zmniejszeniu ulega strumień w szczelinie powietrznej.

Założono, że moment maksymalny jest dla zakresu prędkości $0 \div \omega_{max}/3$, a pozostała charakterystyka to praca na stałą moc. Można uwzględnić przeciążanie silnika w zakresie niskich prędkości bez zwiększania mocy, wówczas proporcjonalnie zwiększa się moment M_{max} i jednocześnie zmniejsza ω_N. Jest to dopuszczalne, ponieważ silniki trakcyjne w zakresie niskich prędkości pracują krótkotrwale z momentem maksymalnym.

W przykładzie założono: $\omega_{max} = 1200$rad/s, $\omega_N = 400$rad/s, $P_N = M_{max}\omega_N$, dlatego parametrem określającym charakterystykę silnika indukcyjnego jest tylko moment. Dodatkowo optymalizacji będzie podlegało przełożenie przekładni mechanicznej i. Uruchomienie optymalizacji odbędzie się przez uruchomienie skryptu `optymalizacja_samochodu.m`, który należy zmienić:

```
przekladnia0=9;
Mmax0=270;
x = fminsearch(@cooper_se,[Mmax0;przekladnia0])%optymalizacja silnika i przekladnii
```

Natomiast funkcja celu zostanie zapisana w pliku `cooper_se.m` i jest to zmieniony program `tesla.m`, ale tutaj w funkcjach celu zadajemy pożądane wartości i dobierana jest charakterystyka silnika oraz przekładnia. Poniżej zamieszczono listing programu liczącego funkcję celu

```
1  function [J4]=cooper_se(x)
2  %OPtymalizacja przekładni mechanicznej
3  % J - to wskaźnik dla przyspieszenia do 100kmh, należy założyć czas
4  % rozpędzania, np.
5  topt=4; %nie krócej niż 4 sek
6  %J1 - to wskaźnik dla 1/4 mili, należy podać optymalny czas, np.
7  topt14=12; % nie krócej niż 12sek
8  %J2 - max. prędkosc, należy podać prędkość pożądaną, np. 320kmh
9  voptmax=200/3.6; %nie szybciej niż, należy pamiętać aby wydłużyć symulację
10 %czas symulacji
11 t_step=0.05; t_end=40;
12 t=0:t_step:t_end;
13
14 Mmax=x(1);
15 przekladnia_i=x(2)
```

```
g=9.81;
%przekladnia_i=8.96;
mass=1365; m2=mass+85;% masa pojazdu z kierowca
d_k=(17*25.4+2*205*0.45)/1000; %srednica kola dla P85
Pn=135000; %Mmax=270;pb=2;
wI=400; %koniec strefy I
%wII=904; % koniec strefy II - dla Coopera jeśli zrealizuje się podbicie
%stałego momentu
wIII=1200; %maksymalna predk. silnika
Pn=Mmax*wI;
w_2k=33.1;
m_k=1.1627e+09;
%-----opory
rho=1.225; % gestość powietrza kg/m3
Cd=0.3; % wsp. oporu powietrza
A=1.98; % płaszczyzna natarcia-konwersja Cd*A=6.2ft^2

%% Przyspieszenie
wm=zeros(1,length(t))+0.01;     %0.01 to wartosc poczatkowa
v_ms=zeros(1,length(t))+0.0001;   %predkosc w m/s
F_g=0*wm;   % nachylenie terenu
clear Me;
dM=12; %load torque in gear
for k=1:length(t)-1
 Me(k)=(wm(k)<=wI)*Mmax+((wm(k)>wI))*Pn/wm(k);
 F_T(k)=2/d_k*przekladnia_i*(Me(k)-dM);   % sila trakcyjna
 F_r(k)=m2*g*0.01*(1+v_ms(k)*3.6/161);
 F_DF(k)=1/2*rho*Cd*A*v_ms(k)^2;
 a(k)=1/m2*(F_T(k)-F_r(k)-F_DF(k)-F_g(k));
 if (wm(k)>wIII) a(k)=0; end
 v_ms(k+1)=v_ms(k)+t_step*a(k); % predkosc w m/s
 wm(k+1)=2*pi*przekladnia_i/(pi*d_k)*v_ms(k);
end
Me(k+1)=Me(k); a(k+1)=a(k);%uzupelnienie o brakujacy element
F_T(k+1)=F_T(k);F_DF(k+1)=F_DF(k);F_r(k+1)=F_r(k);
s_pos=zeros(1,length(t));
for k=2:length(t) % w Matlabie: s_pos=t_step*cumsum(v_ms);
    s_pos(k)=t_step*v_ms(k)+s_pos(k-1);
end;

%%----FUNKCJE CELU:
lt=length(t);   %przyspieszenie do 100km/h
t_end=t(lt);
to=t_end;
n=1;                    %indeksowanie
while(v_ms(n)<100/3.6)&&(n<lt)
    to=t_end;
    n=n+1;
end
to=t(n);
J=(to-topt)^2;  %=========wsk. jakości -- funkcja celu dla 100kmh

% wyznaczenie wskaźnika na 1/4mili=402.5m
lt=length(t);
t_end=t(lt);
t14=t_end;
n=1;                    %indeksowanie
while(s_pos(n)<402.5)&&(n<lt)
    t14=t_end;
    n=n+1;
```

```
76  end
77  t14=t(n);
78  J1=(t14-topt14)^2;
79
80  % wyznaczenie wskaźnika dla max. prędkości
81  lt=length(t);
82  v_max=v_ms(lt);
83  J2=(voptmax-v_max)^2;
84  J3=J+Pn/10000/4;J4=J+Pn/10000/2+J1/3+J2/20;
85  %% wizualizacja
86  figure(5);
87  v_kmh=v_ms*3.6;
88  subplot(211)
89  %p1=plot(t,v_ms,'k',[t(1) t(length(t))], [96.6 96.6]/3.6,'r');yl=ylabel('v/ms');
90  p1=plot(t,v_kmh,'k',[t(1) t(length(t))], [100 100],'r');yl=ylabel('v/kmh');
91  grid; axis tight
92  wynik=sprintf('i=%0.2f,  t_{100}=%0.3f,  v_{max}=%0.1f',przekladnia_i, to,v_max*3.6);
93  title(wynik)
94  set(p1,'LineWidth',1.5)
95  xl=xlabel('t');
96  set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
97  subplot(212)
98  p1=plot(t,s_pos,'k',[t(1) t(length(t))], [402.5 402.5],'r');yl=ylabel('s_{pos}/m');
99  grid; axis tight
100 wynik=sprintf('Pn=%0.1f kW,  M_{max}=%0.0f,  i=%0.2f,   t_{1/4}=%0.3f',Pn/1000, Mmax,przekladnia_i, t14)
101 title(wynik)
102 set(p1,'LineWidth',1.5)
103 xl=xlabel('t');
104 set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
```

1 argumentem jest x, czyli moment maksymalny i przełożenie przekładni (linie 14 i 15), wartości funkcji celu są obliczane w liniach 56÷83 i jest to wartość bieżąca minus wartość zadana (linie 5 ÷ 9), które są podniesione do kwadratu aby uniknąć liczb ujemnych,

40 zmieniono charakterystykę mechaniczną na dwustrefową,

56÷ 83 wyznaczanie funkcji celu zgodnie z opisem w pliku, należy zwrócić uwagę, że wskaźnik $J2$ jest przyjęty jako $-v_{max}$, ponieważ funkcja `fminsearch` poszukuje minimum, to zadanie maksymalizacji prędkości zamienia się na minimalizację,

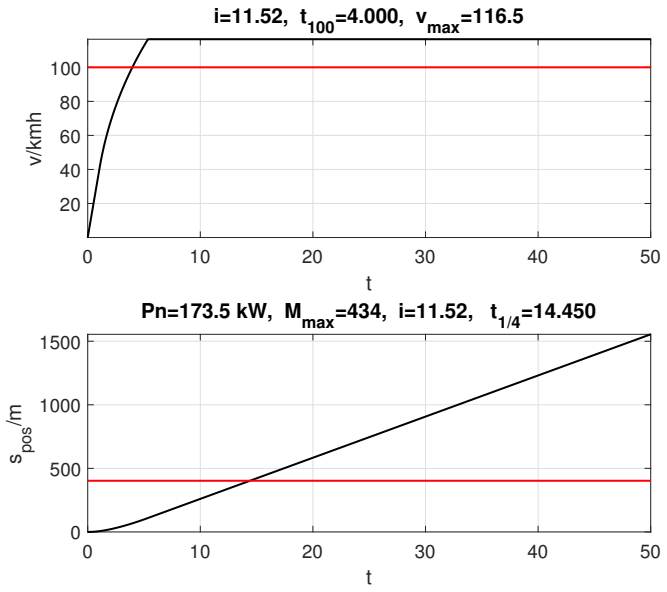
86÷ 104 wizualizacja wyników, dołączono jeszcze wyświetlanie mocy i momentu maksymalnego.

Na rysunku 3.12 przedstawiono wynik optymalizacji dla założonych 4 sekund przyspieszenia do 100km/h warunkiem początkowym było $i(0) = 13$ i moment równy 270Nm. Wyniki te można porównać z optymalizacją dla wartości początkowej $i(0) = 3$ (rys.3.13). Widać, że za każdym razem uzyskane zostało minimum, więc funkcja celu nie jest jednomodalna, a ma wiele minimów. Zwłaszcza różnice w mocy i momencie silników są tu widoczne, dlatego można zaproponować wskaźnik mieszany – polioptymalizacja (optymalizacja wielokryterialna).

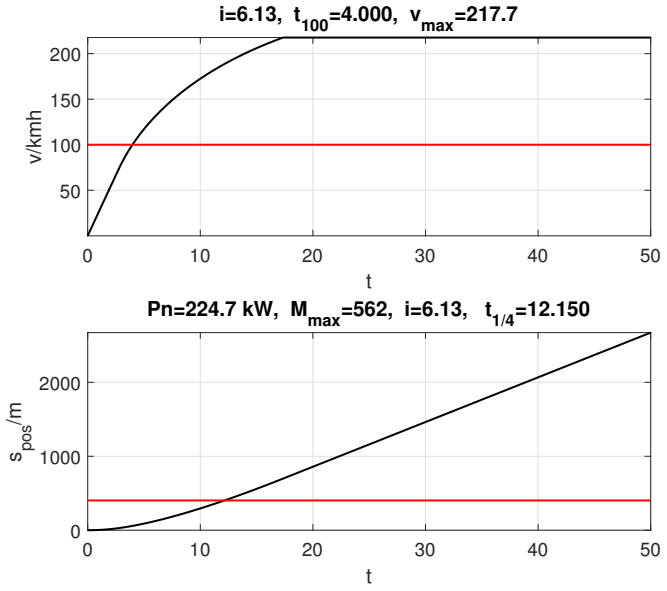
Wprowadza się funkcję celu, której składniki powinny posiadać podobne wartości, ale możemy je zmieniać, np.

$$\mathcal{J}_3 = \alpha_1 \mathcal{J} + \alpha_2 Pn \tag{3.16}$$

gdzie α_i są wagami i można teraz obydwa czynniki znormalizować do jedności, tzn. wartością założonego czasu przyspieszania jest 4s (zmiany wartości wskaźnika jakości mają wartości do kilku), a moc silnika jest wyrażana w tysiącach watów, dlatego można przyjąć: $\alpha_1 = 1,\ \alpha_2 = \frac{1}{2 \cdot 10^5}$. Wówczas czynniki mają podobny zakres zmian i w wyniku optymalizacji uzyskuje się napęd pojazdu, którego osiągi zostały zaprezentowane na rysunku 3.14.



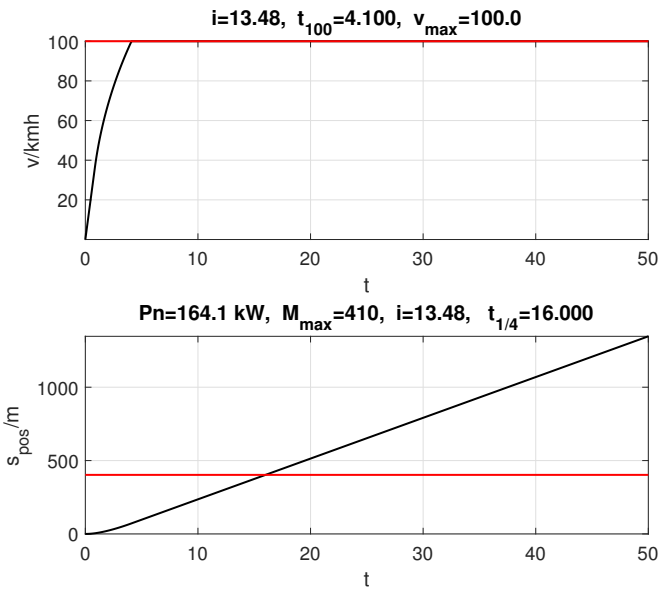
Rysunek 3.12. Osiągi Mini Coopera SE po optymalizacji dla przyspieszenia do 100km/h w 4s, wartość początkowa $i(0) = 13$



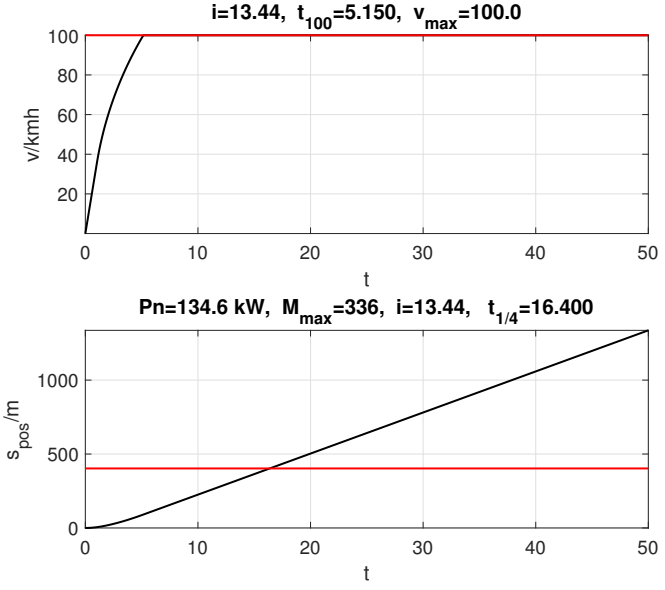
Rysunek 3.13. Osiągi Mini Coopera SE po optymalizacji dla przyspieszenia do 100km/h w 4s, $i(0) = 3$

Dla różnych punktów startowych uzyskuje się zbliżone wyniki. Można teraz zmodyfikować wskaźnik (3.16), tak aby położyć największy nacisk na ograniczenie mocy silnika, ale pozostawić dążenie do przyspieszenia w 4s. W tym celu $\alpha_2 = \frac{1}{10^4}$ i na rysunku 3.15 można zaobserwować znaczne zmniejszenie mocy silnika, ale już czas przyspieszania do 100km/h nie jest bliski 4s. Jest to cena jaką płaci się za ograniczenie mocy.

Przygotowany program można wykorzystywać do wyznaczenia minimum wielokryterialnego, np. zarządać 4s do 100km/h, 12s na 1/4 mili i ograniczenie mocy silnika trakcyjnego. Wówczas wskaźnik



Rysunek 3.14. Osiągi Mini Coopera SE po optymalizacji dla przyspieszenia wg wskaźnika (3.16)

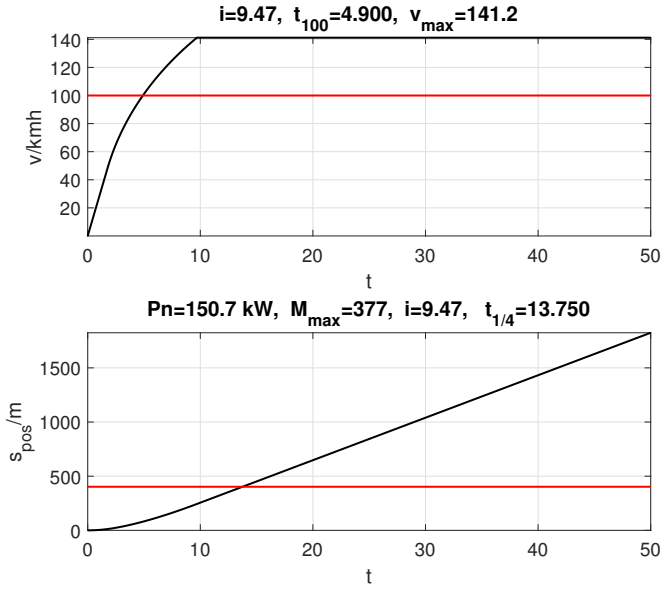


Rysunek 3.15. Osiągi Mini Coopera SE po optymalizacji przekładni dla przyspieszenia do 100km/h w 4 sekundy, wg wskaźnika (3.16), ale największy nacisk położony jest na ograniczenie mocy silnika

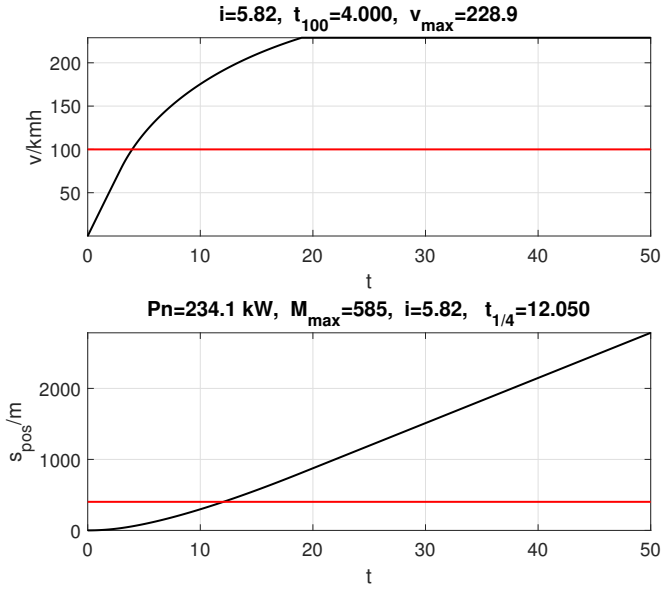
jakości (3.16) należy zapisać jako:

$$\mathcal{J}_4 = \alpha_1 \mathcal{J} + \alpha_2 Pn + \alpha_3 \mathcal{J}_1 \tag{3.17}$$

Dla $\alpha_1 = 1,\ \alpha_2 = 10^{-4},\ \alpha_3 = 1/3$ przedstawiono wyniki na rysunku 3.16. Widać tu, że najsilniej ograniczana jest tu moc, dlatego, że wartości $\alpha_2 Pn$ są największe. Jeżeli zezwoli się na zwiększenie mocy, tj. $\alpha_2 = 10^{-5}$, to założone czasy zostaną uzyskane, ale ograniczenie mocy silnika trakcyjnego praktycznie przestało istnieć.



Rysunek 3.16. Osiągi Mini Coopera SE po dokonaniu optymalizacji przekładni dla przyspieszenia do 100km/h w 4 sekundy, wg wskaźnika (3.17), $\alpha_2 = 10^{-4}$



Rysunek 3.17. Osiągi Mini Coopera SE po dokonaniu optymalizacji przekładni dla przyspieszenia do 100km/h w 4 sekundy, wg wskaźnika (3.17), $\alpha_2 = 10^{-6}$

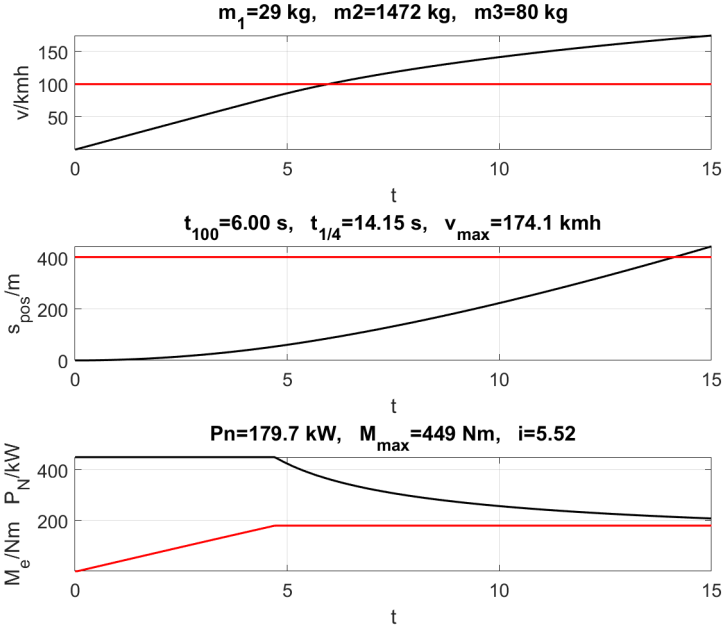
3.1.5. Ćwiczenie

W optymalizacji przyjmuje się zmiany momentu bezwładności silnika trakcyjnego J_1, biorąc pod uwagę czynnik: $\frac{0{,}05\text{kgm}^2}{100\text{kW}}$. Ta wartość dla PMSM jest wyższa niż dla silnika klatkowego w (Sieklucki 2018)i może być przeliczona na masę karoserii m_1 przy pomocy wzoru (3.8).

Druga masa m_2 będzie się również różnić w zależności od mocy silnika trakcyjnego (masa silnika, energoelektroniki i urządzeń elektrycznych) i obejmuje 135kW silnika Cooper SE (współczynnik wynosi $\frac{0{,}5\text{kg}}{1\text{kW}}$), co prowadzi do: $m2 = m_v + (P_N - 135000)/1000 * 0,5$, gdzie m_v to masa samochodu i kierowcy.

Następna masa $m_3 = \sum_i m_{3i}$ jest stała i (3.11) i jest przyjęta jako 80kg. Dlatego w symulacji EV należy użyć całkowitej masy opisanej wzorem (3.11) i przyjąć ją jako 80kg.

Zadanie 1. Wprowadzić zmiany masy opisane powyżej i uzupełnić wizualizację wyniku optymalizacji, tak aby uzyskać wynik jak na rysunku 3.18.



Rysunek 3.18. Ekran wizualizacji

Zadanie 2. Przyjąć funkcję celu dla przyspieszenia do 100km/h:

$$\mathcal{J} = (t_{100} - t_{100opt})^2 \tag{3.18}$$

gdzie t_{100opt} jest pożądanym czasem do 100kmh. Przyjąć $t_{100opt} = 4$s i wykonać optymalizację dla 6 różnych wartości początkowych M_{max} i i. Wartości te powinny umożliwić przeprowadzenie optymalizacji, tzn. jej rozpoczęcie, np. $M_{max} = 10,\ i = 100$ jest źle dobrane–optymalizacja "nie ruszy".

Wyniki zamieścić w tabeli. Widać, że istnieje tu wiele optymalnych rozwiązań, tj. uzyskuje się różne moce, momenty i przełożenia, które spełniają $\mathcal{J} = 0$.

Zadanie 3. Dla różnych warunków początkowych uzyskuje się różne wyniki, a moc silnika wzrasta nawet do 271kW, dlatego konieczna jest modyfikacja funkcji celu, (3.18), np. bierze się pod uwagę moc silnika i to prowadzi do tego:

$$\mathcal{J} = (t_{100} - t_{100opt})^2 + \alpha P_N \tag{3.19}$$

Należy przyjąć $\alpha = 2,5 \cdot 10^{-5}$ oraz 3 inne wartości i przedstawić zestawienie wyników. Określić jaki wpływ ma α na uzyskane wyniki. Czy uzyskano $t_{100opt} = 4$s? Jeśli nie, to ile powinno wynosić t_{100opt} aby uzyskać $t_{100} = 4$s (dla $\alpha = 2,5 \cdot 10^{-5}$)?

Zadanie 4. Optymalizacja dynamicznego, ale nie wyczynowego EV, gdzie przyjmuje się wskaźnik jakości w postaci:

$$\mathcal{J} = (t_{100} - t_{100opt})^2 + \alpha P_N + \beta(t_{1/4} - t_{1/4opt})^2 + \gamma(v_{max} - v_{maxopt})^2 \tag{3.20}$$

Należy przyjąć wartości optymalne $t_{100opt} = 6$s,$t_{1/4opt} = 14$s,$v_{maxopt} = 200$kmh oraz wagi $\alpha = 2,5 \cdot 10^{-5},\ \beta = 0,33,\ \gamma = 0,05$. Pamiętać o wydłużeniu symulacji, np. do 50s. Następnie należy przedstawić i omówić uzyskany rezultat.

3.1.6. Projekt EV

W tej części zostaną podane zasady projektowania samochodu elektrycznego na podstawie istniejącej wersji spalinowej.

Należy przyjąć, że usunięcie napędu wraz z osprzętem to 200kg/2l pojemności skokowej silnika. Składa się na to: silnik z osprzętem, skrzynia biegów, bak z paliwem, alternator, rozrusznik, koło zamachowe.

W projekcie należy uwzględnić:

1. akumulator 10kWh waży 80kg, uwzględnia się tu również system chłodzenia wraz z cieczą chłodzącą oraz wzrost masy zawieszenia wraz ze wzrostem pojemności akumulatora,
2. na działanie systemów pojazdu, które nie są związane z napędem (m.in.klimatyzacja, ogrzewanie, nawiewy, radio, wspomaganie układu kierowniczego) przyjmuje się stałą wartość poboru mocy 1kW. Natomiast system ogrzewania i schładzania akumulatora oraz silnika jest uzależniony od pojemności akumulatora i mocy silnika, przyjmuje się $\frac{0,3\text{kW}}{10\text{kWh}}$ oraz $\frac{0,5\text{kW}}{10\text{kW}}$,
3. wpływ mocy silnika na jego masę, wg zależności$\frac{5\text{kg}}{10\text{kW}}$), współczynnik ten uwzględnia również masę osprzętu elektrycznego (energoelektronika i przewody) oraz systemu chłodzenia. Prowadzi to do zmiany masy pojazdu: $m2 = m_v + P_N/1000 \cdot 0,5$, gdzie m_v to masa samochodu i kierowcy,
4. masa elementów wirujących związanych z kołami $m_3 = \sum_i m_{3i}$ jest stała i (3.11) i jest przyjęta jako 100kg,
5. zmiany momentu bezwładności silnika trakcyjnego J_1, wg współczynnika $\frac{0,05\text{kgm}^2}{100\text{kW}}$ i może być przeliczona na masę karoserii m_1 przy pomocy zależności (3.8).

Przebieg projektu: Dla wybranego pojazdu należy:

1. Policzyć średnicę koła d_w na podstawie danych katalogowych.
2. Należy zaprojektować EV o następujących osiągach: $t_{100opt} = 5$s, $t_{1/4opt} = 13$s, $v_{maxopt} = 190$kmh, o zasięgu autostradowym 300km, przy prędkości 150km/h. Wynikiem będzie: przekładnia, charakterystyka silnika, pojemność akumulatorów (podawać w kWh). Całość doprowadzi do odpowiednich mas m_1, m_2, m_3. Zasięg należy liczyć na podstawie mocy, a nie symulować.
3. Jak zmieni się zasięg pojazdu dla prędkości autostradowej 120, a jak dla 100km/h?

Fajne pojazdy:
`https://www.carfolio.com/renault-megane-renault-sport-275-trophy-368995`

```
https://www.carfolio.com/renault-megane-renaultsport-267817
https://www.carfolio.com/mini-john-cooper-works-countryman-270281
https://en.wikipedia.org/wiki/Tatra_77#cite_note-Tatra_77_aerodynamic_car_czech-4
https://www.carfolio.com/saab-9-3-viggen-81807
https://www.carfolio.com/volkswagen-golf-gti-461516
https://www.carfolio.com/skoda-octavia-rs-245-automatic-516813
https://www.carfolio.com/toyota-yaris-grmn-540287
https://www.carfolio.com/land-rover-discovery-3-4.0-v6-353199
```

Bibliografia

Gillespie, T. (1992), *Fundamentals of Vehicle Dynamics*, Society of Automotive Engineers.

Hindmarsh, J. & Renfrew, A. (1996), *Electrical Machines and Drives*, Oxford, Newnes.

Plunkett, A. & Plette, D. (1977), 'Inverter-induction motor drive for transit cars', *IEEE Transactions on Industry Applications* **13**(1), 26–37.

Sieklucki, G. (2018), 'An investigation into the induction motor of tesla model s vehicle', *2018 international Symposium on Electrical Machines : LIV Sympozjum Maszyn Elektrycznych : Andrychów, Poland, 10?13 June* **dok. electron.**(1). 1-6.

Sieklucki, G. (2021), 'Optimization of powertrain in ev', *Energies* **14**(3).
URL: *https://www.mdpi.com/1996-1073/14/3/725*

Sieklucki, G., Bisztyga, B., Zdrojewski, A., Orzechowski, T. & Sykulski, R. (2014), *Modele i zasady sterowania napędami elektrycznymi*, Kraków, Wydawnictwa AGH.

Dodatek – optymalizacja osiągów i zasięgu EV - pojemność akumulatorów

Korzystając ze wcześniejszych danych, tj. pojemności akumulatorów 32,6kWh=117,36MJ, można wg podanych przeliczników (10kWh/80kg) odjąć od masy m_1 masę akumulatorów i systemu akcesoriów (chłodzenia i grzania): $m_2 = 1365 - \frac{32,6}{10} \cdot 80 = 1365 - 260,8kg \approx 1104$kg.

Następnie należy w funkcji celu dodać dodatkowy argument, którym jest pojemność baterii akumulatorów (`bateria`) i od razu obliczyć jej masę `m4`, którą należy dodać do masy pojazdu `m2`. W kolejnym kroku trzeba przeprowadzić inicjalizację zasięgu `sp` i prędkości autostradowej `vp` i na tej podstawie wyliczyć czas przejazdu `tp`. Znając czas przejazdu i prędkość trzeba wyznaczyć energię potrzebną do pokonania zadanej drogi. W tym celu należy wykorzystać sumę obciążeń drogowych F' podaną wyrażeniem (3.5) i należy określić energię, którą trzeba z jednostek J przeliczyć do kWh.

Teraz należy napisać nowy wskaźnik jakości, w którym porównane zostanie zapotrzebowanie na energię z pojemnością baterii akumulatorów. Należy również pamiętać o podaniu wartości początkowej pojemności baterii przed wywołaniem funkcji optymalizującej.

A wszystko rozwiązuje funkcja celu w postaci:

```
function [J5]=cooper_se3(x)
%OPtymalizacja przekładni mechanicznej i momentu przy zmianach masy od mocy
%silnika i zadanym zasięgu=wyznacza pojemność baterii
% J - t100 wskaźnik dla przyspieszenia do 100kmh, należy założyć czas
% rozpędzania, np.
t100pt=5; %nie krócej niż 4 sek
%J1 - t100 wskaźnik dla 1/4 mili, należy podać optymalny czas, np.
t14opt=13; % nie krócej niż 12sek
%J2 - max. prędkosc, należy podać prędkość pożądaną, np. 320kmh
voptmax=190/3.6; %nie szybciej niż, należy pamiętać aby wydłużyć symulację
%czas symulacji
%J3 uwzględnienie mocy silnika
%J5 oblicza dodatkowo zasięg autostradowy
```

```
14
15 vp=150/3.6; %prędkość autostradowa
16 sp=300*1000; % odległość jazdy po autostradzie
17 tp=sp/vp; % czas przejazdu po autostradzie
18
19
20 t_step=0.05; t_end=50;
21 t=0:t_step:t_end;
22
23 Mmax=x(1);
24 przekladnia_i=x(2);
25 bateria=x(3); %kWh
26 m4=x(3)*8; %masa baterii akumulatorów
27
28 g=9.81;
29 %przekladnia_i=8.96;
30 d_k=(17*25.4+2*205*0.45)/1000; %srednica kola
31
32 wI=400; %koniec strefy I
33 %wII=904; % koniec strefy II - dla Coopera jeśli zrealizuje się podbicie
34 %stałego momentu
35 wIII=1200; %maksymalna predk. silnika
36 Pn=Mmax*wI;
37 m2=1104+85+m4;% masa pojazdu z kierowca
38 m2=m2+(Pn-135000)/1000*0.5;
39 JJ1=Pn/100000*0.05;%mom. bezwładnosci: 0.05kgm2 na każde 100kW
40 m1=JJ1*4*przekladnia_i^2/(d_k^2);
41 m3=80;%waga kół
42 mass=m1+m2+m3;
43 w_2k=33.1;
44 m_k=1.1627e+09;
45 %-----opory
46 rho=1.225; % gest100ść powietrza kg/m3
47 Cd=0.3; % wsp. oporu powietrza
48 A=1.98; % płaszczyzna natarcia-konwersja Cd*A=6.2ft^2
49
50 %% Przyspieszenie
51 wm=zeros(1,length(t))+0.01;     %0.01 t100 wart100sc poczatkowa
52 v_ms=zeros(1,length(t))+0.0001;   %predkosc w m/s
53 F_g=0*wm;   % nachylenie terenu
54 clear Me;
55 dM=12; %load t100rque in gear
56 for k=1:length(t)-1
57  Me(k)=(wm(k)<=wI)*Mmax+((wm(k)>wI))*Pn/wm(k);
58  F_T(k)=2/d_k*przekladnia_i*(Me(k)-dM);   % sila trakcyjna
59  F_r(k)=m2*g*0.01*(1+v_ms(k)*3.6/161);
60  F_DF(k)=1/2*rho*Cd*A*v_ms(k)^2;
61  a(k)=1/mass*(F_T(k)-F_r(k)-F_DF(k)-F_g(k));
62  if (wm(k)>wIII) a(k)=0; end
63  v_ms(k+1)=v_ms(k)+t_step*a(k); % predkosc w m/s
64  wm(k+1)=2*pi*przekladnia_i/(pi*d_k)*v_ms(k);
65 end
66 Me(k+1)=Me(k); a(k+1)=a(k);%uzupelnienie o brakujacy element
67 F_T(k+1)=F_T(k);F_DF(k+1)=F_DF(k);F_r(k+1)=F_r(k);
68 s_pos=zeros(1,length(t));
69 for k=2:length(t) % w Matlabie: s_pos=t_step*cumsum(v_ms);
70     s_pos(k)=t_step*v_ms(k)+s_pos(k-1);
71 end;
72
73 %%----FUNKCJE CELU:
```

```
lt=length(t);   %przyspieszenie do 100km/h
t_end=t(lt);
t100=t_end;
n=1;                    %indeksowanie
while(v_ms(n)<100/3.6)&&(n<lt)
    t100=t_end;
    n=n+1;
end
t100=t(n);
J=(t100-t100pt)^2;  %=========wsk. jakości -- funkcja celu dla 100kmh

% wyznaczenie wskaźnika na 1/4mili=402.5m
t14=t_end;
n=1;                    %indeksowanie
while(s_pos(n)<402.5)&&(n<lt)
    t14=t_end;
    n=n+1;
end
t14=t(n);
J1=(t14-t14opt)^2;

% wyznaczenie wskaźnika dla max. prędkości
v_max=v_ms(lt);
J2=(voptmax-v_max)^2;

J3=J+Pn/10000/1;
J4=J+Pn/10000/0.3+J1/3+J2/20;

F_p=m2*g*0.01*(1+vp*3.6/161)+1/2*rho*Cd*A*vp^2; % siła wypadkowa
E_p=F_p*vp*tp/(1000*3600); % Energia ruchu w kWh
J5=J+Pn/10000/1+J1/3+J2/20+(bateria-E_p)^2/10;
%% wizualizacja
figure(5);
v_kmh=v_ms*3.6;
subplot(411)
%p1=plot(t,v_ms,'k',[t(1) t(length(t))], [96.6 96.6]/3.6,'r');yl=ylabel('v/ms');
p1=plot(t,v_kmh,'k',[t(1) t(length(t))], [100 100],'r');yl=ylabel('v/kmh');
grid; axis tight
wynik=sprintf('m_1=%0.0f kg,   m2=%0.0f kg,   m3=%0.0f kg, m4=%0.0f kg',m1, m2,m3,m4);
title(wynik)
set(p1,'LineWidth',1.5)
xl=xlabel('t');
set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
subplot(412)
p1=plot(t,s_pos,'k',[t(1) t(length(t))], [402.5 402.5],'r');yl=ylabel('s_{pos}/m');
grid; axis tight
wynik=sprintf('t_{100}=%0.2f s,   t_{1/4}=%0.2f s,   v_{max}=%0.1f kmh',t100, t14, v_max*3.6);
title(wynik)
set(p1,'LineWidth',1.5)
xl=xlabel('t');
set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
subplot(413)
p1=plot(t,Me,'k',t,Me.*wm/1000,'r');yl=ylabel('M_e/Nm   P_N/kW');
grid; axis tight
wynik=sprintf('Pn=%0.1f kW,   M_{max}=%0.0f Nm,   i=%0.2f',Pn/1000, Mmax,przekladnia_i);
title(wynik)
set(p1,'LineWidth',1.5)
xl=xlabel('t');
set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
subplot(414)
```

```
txx=0:1:tp;
p1=plot(txx,F_p*vp*txx/(1000*3600),'k');yl=ylabel('Energia');
grid; axis tight
wynik=sprintf('Pn=%0.1f kW,   M_{max}=%0.0f Nm,   bat.=%0.2f kWh',Pn/1000, Mmax,bateria);
title(wynik)
set(p1,'LineWidth',1.5)
xl=xlabel('t');
set(xl,'FontSize',16);set(yl,'FontSize',15);set(gca,'FontSize',14);
```

Oraz wywołanie procedury optymalizacji Simplex dla tego problemu:

```
%optymalizacja samochodu
przekladnia0=9;
Mmax0=500;
bateria=50; % podane w kWh
%[i] = fminsearch(@tesla,przekladnia0)%optymalizacja przekłdni mech.
%x = fminsearch(@cooper_se,[Mmax0;przekladnia0])%optymalizacja silnika i przekladnii
%optymalizacja silnika i przekladnii z uwzględnieniem zmiany masy od mocy
%silnika
%x = fminsearch(@cooper_se2,[Mmax0;przekladnia0])
x = fminsearch(@cooper_se3,[Mmax0;przekladnia0;bateria])
```